

Residual-Guided Hypothesis Language Induction for Scientific Reasoning with Small Language Models

Anonymous submission

Abstract

Scientific reasoning with small language models is limited not only by final answer verification, but by the earlier problem of constructing the right hypothesis space before the answer is known. Recent agentic methods improve long-horizon behavior by adding tool use, reflection, search, and executable actions. But a structural gap remains: failures are usually stored as text or trajectory history, so the system does not explicitly change the hypothesis language used to generate the next scientific artifact. We address this with two ideas. First, every task is represented through a shared typed hypothesis state containing answer slots, evidence contracts, executable artifacts, validators, and residuals. Second, repeated validation failures are converted into typed residual classes, from which candidate operators are induced and promoted only when they improve held-out tasks after a complexity penalty. We implement these ideas as RG-HLI, a residual-guided hypothesis-language induction system around a GPT-4o-mini core. The same state, scoring, residual, and promotion interface is used on DiscoveryBench, NewtonBench, and UltraHorizon. Complete runs reach 28.09 HMS on DiscoveryBench, 49.7% audited symbolic accuracy on NewtonBench, and 75.36 score on UltraHorizon. Controlled ablations show that typed contracts, answer-slot closure, protocol satisfaction, and residual-conditioned operator promotion explain distinct parts of the gain. The results support a narrow mechanism-level claim: small models reason more effectively when hypothesis construction is externalized as typed, executable, residual-aware language growth rather than sampled only as free-form prose.

1 Introduction

Scientific discovery tasks ask an agent to do more than answer a question. The agent must decide what should be measured, what kind of relation is plausible, which variables play stable roles, and when a proposed explanation has been falsified. Recent benchmarks make this distinction explicit. DiscoveryBench casts data-driven discovery as hypothesis search over real datasets and metadata (Majumder et al. 2024). NewtonBench evaluates interactive scientific law discovery rather than static curve fitting (Zheng et al. 2025). UltraHorizon stresses long-horizon partially observable exploration (Luo

et al. 2025). Across these settings, the bottleneck is often not final verification but the construction of a useful hypothesis space before the answer is known.

This bottleneck is especially sharp for small language models. Standard inference-time scaffolds extend the trajectory: REACT interleaves reasoning and actions (Yao et al. 2023b), Reflexion and Self-Refine add verbal feedback (Shinn et al. 2023; Madaan et al. 2023), Tree-of-Thoughts and LATS search over reasoning branches (Yao et al. 2023a; Zhou et al. 2024a), and CodeAct uses executable code as the action interface (Wang et al. 2024). These methods improve many agent tasks, but they still largely rely on the model to propose the intermediate concepts that make the search fruitful. If the model never represents the task in the right coordinates, more attempts can simply explore a larger set of wrong programs.

RG-HLI starts from a different premise: hypothesis generation should be treated as an external, executable induction problem, not as a longer natural-language trace. A scientific hypothesis is represented as an artifact with typed holes, evidence contracts, executable measurements, validation tests, and residuals. The language model proposes local moves, but the environment closes uncertain parts whenever possible. For example, instead of asking the model to guess a complete law or discovery statement, the system asks which roles must be filled, which measurement can identify each role, which residual remains after execution, and which operator would make that residual simpler on future tasks.

The central object in RG-HLI is a shared hypothesis state, `HYPOTHESISCONTEXT`. It is read and written by heterogeneous components: a universal hypothesis kernel, evidence-contract compiler, estimand synthesizer, role canonicalizer, coordinate-probe engine, metamorphic verifier, and residual compiler. This is not a transcript memory. It is a typed state over hypotheses: what has been measured, which slots have been closed, which answer forms are admissible, which residuals recur, and which executable operators compress previous failures. In this respect RG-HLI is closer in spirit to library-learning systems such as DreamCoder (Ellis et al. 2021) than to a pure agent loop, but its learned unit is not a domain-specific program alone. It is a reusable way of turning an underspecified scientific task into measurable hypothesis objects. In ReAct-style agents, a failed tool result remains contextual feedback to the model; in RG-HLI, it is

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

mapped to a typed residual whose type constrains the next admissible executable operators.

The contribution is threefold. First, we formulate small-model scientific reasoning as *typed executable hypothesis induction*: a sequence of typed measurements, validations, and residual-conditioned language edits. Second, we instantiate this formulation as one shared state, scoring, residual, and promotion interface that is instantiated with interface-specific executable operator families for tabular discovery, symbolic law induction, and long-horizon rule discovery. Third, we report full-run evaluations and mechanism ablations that separate fixed-language executable search from residual-conditioned language growth. The empirical results show that externalizing hypothesis generation makes failures local and diagnosable, while also revealing a clear remaining bottleneck: small models still need richer mechanisms for generating useful hypothesis representations when the missing object cannot be reduced to a measurement.

The paper tests a specific mechanism-level hypothesis: small models fail scientific reasoning tasks when intermediate hypotheses remain untyped strings, and they improve when those hypotheses become executable objects with typed slots, validators, and residuals. This makes the claim narrower than general “agent improvement” and broader than a benchmark solver: RG-HLI is an interface for changing the hypothesis language available to the model at inference time.

2 Preliminaries

We write a task instance as

$$x = (q_x, D_x, \mathcal{T}_x, \mathcal{A}_x, S_x). \quad (1)$$

Here q_x is the natural-language instruction, D_x denotes observations such as tables, metadata, traces, or simulator state, $\mathcal{T}_x$ is an optional tool or environment interface, $\mathcal{A}_x$ is the admissible answer space, and $S_x : \mathcal{A}_x \rightarrow \mathbb{R}$ is the benchmark evaluator. The system observes $q_x, D_x, \mathcal{T}_x$, and the admissible output format, but does not observe the gold answer before producing its response.

A candidate answer may be a prose hypothesis, a symbolic law, a program, a transition rule, or a structured report. We use $h \in \mathcal{H}_x$ for such a scientific artifact and $R_x(h) \in \mathcal{A}_x$ for its rendered benchmark answer. The distinction is useful because an artifact may contain intermediate objects—variables, relations, measurements, code, or evidence—that are not directly shown in the final answer string.

Scientific tasks usually constrain the form of a valid artifact. We denote the task-level constraints by

$$C_x = \{s_i = (\tau_i, d_i, \nu_i)\}_{i=1}^k, \quad (2)$$

where τ_i is a slot type, d_i describes the required evidence or answer component, and ν_i is a local validity predicate when one is available. These constraints describe the task, not any particular solver. They specify which parts of an artifact must be grounded before the rendered answer can be considered valid.

Let $E_x(h)$ denote evidence that can be obtained from the observed interface: statistics computed from a dataset, a fitted symbolic expression, a simulator trace, a tool result, or

a replay check. When an artifact violates a constraint or fails an available check, we write the discrepancy as a residual

$$\rho = (\tau_\rho, \ell_\rho, c_\rho, e_\rho, v_\rho), \quad (3)$$

where τ_ρ is the residual type, ℓ_ρ is its location in the artifact or trace, $c_\rho \in C_x$ is the relevant constraint, e_ρ is the supporting evidence, and v_ρ is the failed check or validator. This notation only records what failed; later sections describe how residuals are used by the proposed system.

A hypothesis language is a finite typed language

$$L = (\mathcal{Y}, \mathcal{O}), \quad (4)$$

where $\mathcal{Y}$ is a set of artifact and slot types, and $\mathcal{O}$ is a set of executable typed operators. For a task x , the language induces the search space $\mathcal{H}_x(L) \subseteq \mathcal{H}_x$ of artifacts that can be constructed and checked using those types and operators.

We use $\mathcal{M}$ for an abstract hypothesis state: a record of the current contract, candidate artifacts, closed slots, evidence, and residuals. At this level it is only a notational object. A language operator is any typed executable transformation of the form

$$O : \mathcal{M} \times x \rightarrow \mathcal{M}. \quad (5)$$

The concrete state representation and operator interface used by RG-HLI are defined in Section 4.

Language induction denotes a sequence of language states

$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_T, \quad L_{t+1} = L_t \cup \{o_t^*\}, \quad (6)$$

where each added operator expands the set of artifacts that can be constructed on later tasks. This notation does not specify how o_t^* is chosen; the selection mechanism is part of the method.

The objective on each instance is to return $a = R_x(h)$ with high benchmark score $S_x(a)$. Equivalently, a scientific reasoning system must instantiate the chain

$$x \rightarrow C_x \rightarrow h \in \mathcal{H}_x(L) \rightarrow R_x(h) \rightarrow S_x(R_x(h)). \quad (7)$$

The benchmark evaluator scores only the rendered answer, but the intermediate objects determine whether a small model can construct a useful answer before seeing the gold target.

3 Related Work

Reasoning-time scaffolds for LLM agents. Chain-of-thought prompting showed that exposing intermediate reasoning can improve large-model problem solving (Wei et al. 2022). Later inference-time methods make this computation more structured. Self-consistency marginalizes over sampled reasoning paths (Wang et al. 2023b); least-to-most prompting decomposes difficult problems into easier subproblems (Zhou et al. 2023); REACT couples reasoning traces with actions (Yao et al. 2023b); Reflexion and Self-Refine use feedback to revise later attempts (Shinn et al. 2023; Madaan et al. 2023); Tree-of-Thoughts and LATS make the search tree explicit (Yao et al. 2023a; Zhou et al. 2024a); STaR turns successful generated rationales into training data (Zelikman et al. 2022); and CodeAct unifies tool use around executable Python actions (Wang et al. 2024). These methods are closest to RG-HLI at the level of execution and iterative correction, but they do not by themselves define what a scientific

hypothesis object is. RG-HLI uses the same broad agentic insight–interleave model calls with external computation–but makes the persistent unit a typed hypothesis state rather than a reasoning transcript. This distinction matters when the model’s blind spot is representational. If a model does not propose the right variables, coordinates, answer form, or transition clauses, additional sampled traces can preserve the same missing object. RG-HLI instead makes the missing object part of the state: a hole or residual that later operators can target.

Self-optimizing language-model pipelines. DSPy treats LM programs as optimizable computational graphs rather than hand-written prompt strings (Khatab et al. 2023). Self-Discover asks a model to compose reasoning structures before solving a task (Zhou et al. 2024b). Voyager stores executable skills learned through interaction with Minecraft (Wang et al. 2023a). PAL and Program-of-Thoughts offload computation to a program runtime once the model has produced executable intermediate steps (Gao et al. 2023; Chen et al. 2023). Toolformer shows that tool-use decisions themselves can be learned from self-supervised signals (Schick et al. 2023). These systems show that the right unit of adaptation may be a program, module, executable trace, tool policy, or reusable skill. RG-HLI differs in two ways. First, its stored unit is tied to scientific evidence: contracts, measurements, residuals, and validators. Second, promotion is residual-conditioned; an operator is useful only if it closes typed holes or compresses recurrent failures across tasks.

Program induction and library learning. DreamCoder learns a domain language and neural search policy through a wake-sleep loop over solved and imagined programs (Ellis et al. 2021). Later library-learning work further explores how abstractions reduce search breadth and depth, while FunSearch demonstrates that LLM-guided evolutionary program search can produce new mathematical constructions when candidate programs are externally evaluated (Romera-Paredes et al. 2024). This line is a major conceptual ancestor of RG-HLI: these systems treat expertise as a changing language rather than a fixed prompt. The difference is that RG-HLI operates in mixed scientific interfaces where the answer may be prose, a law, or a transition program, and where holes can often be closed by measurements rather than by full program synthesis.

Symbolic regression and scientific law discovery. Symbolic regression recovers compact formulas from data. AIFeynman exploits physics-inspired structure such as symmetry and separability (Udrescu and Tegmark 2020); PySR provides a practical evolutionary symbolic-regression engine for scientific modeling (Cranmer 2023). NewtonBench moves beyond static formula fitting by requiring interactive rediscovery of shifted physical laws (Zheng et al. 2025). RG-HLI borrows the insistence on executable hypotheses, but does not assume that every task reduces to equation recovery. Its coordinate probes are one operator family inside a broader hypothesis-state kernel.

Causal and experimental-design perspectives. Scientific hypotheses are not merely predictors; they make claims about

Table 1: Conceptual comparison to DreamCoder-style library learning.

Aspect	DreamCoder-style library learning	RG-HLI
Abstraction source	solved programs	typed validation residuals
Learned unit	DSL function / library primitive	operator over hypotheses, evidence, and renderers
Promotion signal	compression and synthesis success	held-out gain minus complexity
Object of coverage	program behavior	missing roles, contracts, validators, and traces
Output type	programs	prose hypotheses, laws, or transition artifacts

stable relations under interventions, environments, or perturbations. Structural causal models formalize this distinction (Pearl 2009), invariant prediction uses stability across environments to identify causal structure (Peters, Bühlmann, and Meinshausen 2016), and Bayesian experimental design studies how to choose measurements that maximize information (Rainforth et al. 2023). RG-HLI is not a causal discovery algorithm, but it adopts the same discipline: a hypothesis must state what evidence would support it, what transformation should preserve it, and what residual would falsify or refine it.

Automated scientific-discovery agents. Recent systems aim to automate larger pieces of the research lifecycle. The AI Scientist frames end-to-end machine-learning research as an agentic loop over idea generation, code, experiments, analysis, and writing (Lu et al. 2024; Yamada et al. 2026). AutoBench evaluates scientific discovery as interactive causal-structure recovery (Chen et al. 2025), while AutoDiscovery uses Bayesian surprise to prioritize open-ended data-driven hypotheses (Agarwal et al. 2025). These works motivate the ambition of autonomous discovery, but they also underline a central risk: if the system’s adaptation is only an outer agent loop, failures remain difficult to localize. RG-HLI addresses a narrower mechanism: the unit that changes over time is the typed hypothesis language used to generate and validate candidate scientific artifacts.

Benchmarks for scientific agents. DiscoveryBench, NewtonBench, and UltraHorizon expose different faces of the same problem: the agent must build an internal research object before it can answer. DiscoveryBench emphasizes data and metadata alignment (Majumder et al. 2024); NewtonBench emphasizes interactive law induction and shifted physical laws (Zheng et al. 2025); UltraHorizon emphasizes long-horizon memory, tool use, and partial observability (Luo et al. 2025). We use these benchmarks not as separate engineering targets but as probes of whether one hypothesis-state representation can support multiple scientific task interfaces.

4 RG-HLI

4.1 Overview

RG-HLI is a hypothesis-induction kernel around a small language model. The model is used to propose typed sketches, measurements, and repairs, but the state of the search is external and executable. The same loop is used for all benchmarks: compile a contract, generate artifacts, execute measurements, validate them, store residuals, and occasionally promote a residual pattern into a new operator. Operators do not pass only text to later calls; they update one typed context containing contracts, artifacts, evidence, residuals, and promoted operators.

The design is governed by three invariants. *Typed state*: every useful intermediate claim must be represented as a slot, artifact, validator, or residual rather than only as prose. *Executable closure*: whenever a slot can be identified by a computation or probe, the system closes it by execution instead of asking the model to guess it again. *Residual discipline*: a failed hypothesis can expand the operator language only when the residual type recurs and the proposed operator improves held-out tasks after a complexity penalty. These invariants are the difference between RG-HLI and a longer self-refinement transcript.

The supplement instantiates this mechanism on a concrete language update: a failed transition artifact becomes a typed residual, several similar residuals form an ordered-threshold residual class, and the induced operator expands the next hypothesis language.

4.2 The HYPOTHESISCONTEXT

For a task x , the context is

$$\mathcal{M}_x = (C, \mathcal{H}, \Theta, \mathcal{E}, \mathcal{R}, \mathcal{O}, h^*), \quad (8)$$

where C is the evidence contract, $\mathcal{H}$ is the pool of candidate artifacts, Θ stores closed typed holes, $\mathcal{E}$ stores executable evidence, $\mathcal{R}$ stores typed residuals, $\mathcal{O}$ is the active operator set, and h^* is the current best answer artifact. All operators read and write this object. This is the main architectural constraint: every stage must return a state update, not a private chain of thought.

Each operator $O_j \in \mathcal{O}$ has the same interface,

$$O_j(\mathcal{M}_x, x) \rightarrow \{(h, e, \rho, \Delta\mathcal{M})\}, \quad (9)$$

where h is a proposed artifact, e is executed evidence, ρ is the typed residual, and $\Delta\mathcal{M}$ is the state update. This interface is what makes the system benchmark-independent. A tabular estimand, a symbolic coordinate probe, and a transition-rule inducer differ internally, but the kernel only sees artifact, evidence, residual, and update.

Candidate scoring and typed residuals. The kernel ranks candidates by a shared energy score:

$$\begin{aligned} \text{score}(h) = & \lambda_E E(h, x) + \lambda_C \text{cover}(h, C) \\ & - \lambda_V V(h, C) - \lambda_K K(h). \end{aligned} \quad (10)$$

This objective is Bayesian in motivation—evidence should increase confidence, while instability and complexity should decrease it—but we do not require a calibrated likelihood

Table 2: Energy terms used by the shared hypothesis kernel.

Term	Definition	Range	Weight
E	executable fit or local score	$[0, 1]$	24
Cover	contract slots covered	$[0, 1]$	34
V	metamorphic validation loss	$[0, 1]$	32
K	operator complexity	≥ 0	2

or learned prior. The operational object is the benchmark-independent energy decomposition and the typed residual emitted when that score fails. Table 2 makes the scoring parameterization explicit. The same decomposition is used across interfaces, while each operator normalizes its executable evidence to a comparable range before entering the shared ranking function. The weights were fixed for the reported runs; sensitivity to these constants remains a target for a larger calibration study.

End-to-end trace. In a NewtonBench-style trajectory, the model does not guess the law directly: the contract asks for coordinate, exponent, constant, and residual slots; a sketch $y = c\phi(x)^p$ is executed; residual curvature activates a coordinate probe; the probe closes the coordinate, exponent, and constant slots; and the held-out validator accepts the artifact. The failed first probe changes typed state rather than merely changing the next prompt.

4.3 Operators

Contract compiler. The contract compiler maps the input interface to required slots and local validators. It does not predict the gold answer. It predicts what any valid answer must make observable: answer type, scope, variables, measurement target, relation form, and rendering constraints.

Typed-hole closure. Instead of asking the model for a complete hypothesis, RG-HLI asks for sketches with typed holes and then closes the holes by small executable probes whenever possible. For a tabular question, a hole may be an outcome role or a window operator. For a law-discovery task, it may be a coordinate chart or exponent. For a sequence task, it may be a state variable or transition clause. Closing a hole adds an assignment to Θ and reduces the search space for later holes.

Executable estimands and coordinate probes. The estimand operator creates statistical objects for tabular discovery: group contrasts, interaction effects, coefficient shifts, multi-item proportions, and time-window measurements. The coordinate-probe operator creates compact transformations for laws and rules: log-log fits, affine and polynomial lifts, ratios, finite differences, thresholds, and simple state-transition clauses. Both operators produce evidence objects rather than only prose rationales.

Metamorphic verifier. The verifier checks whether the artifact answers the requested question under transformations that should preserve the object: renaming irrelevant columns, holding answer form fixed, changing display order, perturbing nuisance values, or replaying held-out traces.

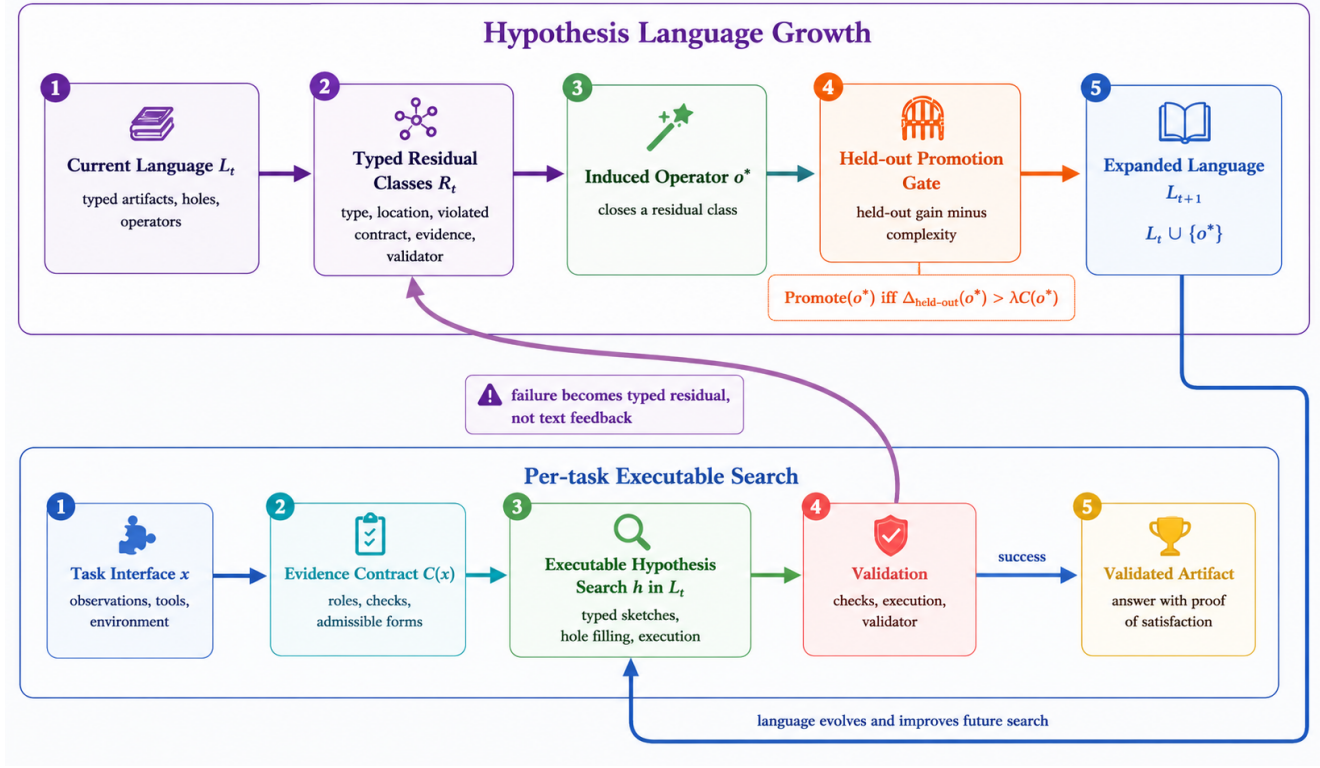


Figure 1: Hypothesis-language growth in RG-HLI. The lower loop performs per-task executable search: a task interface is compiled into evidence contracts, searched in the current hypothesis language, and validated as an artifact. Failures are not kept as free-form feedback; they become typed residual classes. The upper loop turns recurring residual classes into candidate operators and promotes them only when held-out gain exceeds complexity, yielding an expanded language for future tasks.

These checks are label-free and therefore available before benchmark evaluation.

4.4 Induction Algorithms

The procedure runs at two time scales. At benchmark time, the language L^* is frozen. The system compiles C_x , initializes $\mathcal{M}_x$, repeatedly selects admissible operators, executes their probes, writes artifacts/evidence/residuals back into the context, updates h^* by the shared score, and finally renders $R_x(h^*)$. Residuals from reported test rows may be logged for analysis, but they cannot change L^* for later reported rows. At development time, residuals accumulated on development tasks are clustered by type-compatible fields and evidence signature. Each cluster may propose a candidate operator through the same small-model code-and-sketch interface used for task solving. The operator is discarded unless it passes static type checks, sandbox execution, validators, and the held-out promotion rule below. Full pseudocode is provided in the supplement.

Language-growth protocol. The promotion step has two modes. During development, residuals may induce candidate operators, which are promoted only if they improve a held-out development set after the complexity penalty in Eq. (11). For the headline benchmark runs, the resulting operator language L^* is frozen before the reported evaluation. The benchmark

score is never used as the promotion signal for examples later reported as test results. Online adaptation is therefore limited to per-task hypothesis search in the frozen language, not to changing the language using future test labels.

4.5 Residual-Conditioned Operator Promotion

The residual-extension step is not arbitrary code growth. A residual cluster $R = \{\rho_1, \dots, \rho_m\}$ can propose an operator only if it identifies a repeatable missing transformation. Formally, a candidate operator O' is promoted when

$$\Delta(O') = \mathbb{E}_{x \sim \mathcal{D}_{\text{held}}} [S(h_{O'}^*(x)) - S(h^*(x))] - \beta K(O') > 0, \quad (11)$$

and the improvement is not confined to the task that produced the residual. This rule is meant to prevent an ever-growing bag of benchmark-specific tricks: an operator must improve held-out tasks under the same contract and scoring interface while paying a complexity penalty.

4.6 Why This Helps Small Models

RG-HLI changes the role of the language model. The model does not need to hold the entire long-horizon proof in context or invent the final hypothesis in one sample. It repeatedly proposes local typed objects whose consequences can be executed. Each closed hole constrains later holes, each verifier rejects an invalid answer form before final scoring, and each

residual becomes a structured training signal for the outer loop. The intended gain is therefore not more verbal reasoning, but a better external search geometry for capacity-limited models.

5 Experimental Evaluation

5.1 Benchmarks and protocol

We evaluate on three benchmark families that stress different scientific interfaces while sharing the same central difficulty: the agent must construct a hypothesis object before it can answer. DiscoveryBench provides real datasets, metadata, and natural-language discovery goals; the final answer is scored by Hypothesis Matching Score (HMS) and a consistency-HMS variant. NewtonBench provides interactive scientific-law discovery tasks in shifted physical worlds; we report symbolic accuracy over all tasks (SA-all) and over non-abstained answers (SA-answered). UltraHorizon provides long-horizon partially observable rule-induction environments; we report the reproduced paper-style environment score.

The reported system uses a GPT-4o-mini core throughout. We use one shared hypothesis-induction stack: contract compilation, typed-hole closure, estimand synthesis, coordinate probing, metamorphic validation, and residual-conditioned promotion. Benchmark adapters expose observations and accept final artifacts; interface-specific operator families expose measurable objects appropriate to each environment, but they all write the same `HYPOTHESISCONTEXT` and are scored by the same evidence, validation, residual, and complexity rules. We exclude diagnostic ceiling runs that used task-family measurement bootstraps or hand-engineered rendering fixes.

Reproducibility audit. For DiscoveryBench we evaluate the repository real-test task rows using the benchmark evaluator, producing 239 predictions and 239 scored rows. For NewtonBench we evaluate the full cross-product of 12 physics domains, three difficulty levels, three law versions, and three system variants, for 324 symbolic-law tasks. For UltraHorizon we evaluate 32 seeds for each of the three official environments, for 96 total runs, using the paper-style judge, hard difficulty, 50-step episodes, and no environment hints or measurement bootstraps. The row-level outputs are retained for each benchmark, and the main table is generated only from complete runs with the expected row counts. The language used in these evaluations is frozen before scoring; residuals from reported test rows are analyzed after the run and are not used to promote new operators for later rows in the same headline evaluation. For UltraHorizon we report both a strict agent-only ablation and the full RG-HLI system. The strict run disables fallback commits, environment hints, and measurement bootstraps. The full run adds a terminal protocol controller: it uses only public environment actions to satisfy explicit terminal preconditions, such as the Bio environment’s 50-successful-cross requirement, before compiling the final artifact from the observed trace. It does not inspect hidden rules, gold labels, or judge feedback.

Table 3: Benchmark interfaces and metrics.

Benchmark	Interface	Artifact	Metric
DiscoveryBench	tables, metadata, goal	hypothesis	HMS / Cons-HMS
NewtonBench	simulator, observations	symbolic law	SA-all / SA-answered
UltraHorizon	POMDP-style env.	transition rule	score

5.2 Main full-run results

Table 4 reports complete benchmark runs under native metrics. The same GPT-4o-mini-core kernel is competitive with published prompt-level references on DiscoveryBench, slightly above the o4-mini reference on NewtonBench, and strong on UltraHorizon once the public terminal protocol is satisfied. We interpret the table as mechanism evidence rather than universal leaderboard dominance: the operator language is frozen, the benchmark score is not used for test-time promotion, and larger frontier systems remain stronger on NewtonBench.

Table 4: Full-run GPT-4o-mini-core RG-HLI results.

Benchmark	Tasks	Result	Secondary detail
DiscoveryBench	239	28.09 HMS	36.04 consistency-HMS
NewtonBench	324	49.7% SA	67.1% SA-answered
UltraHorizon	96	75.36	full: Grid 54.38; Seq 81.25; Bio 90.47
UltraHorizon	96	51.04	strict: Grid 59.38; Seq 93.75; Bio 0.00

Table 5: Published references under native metrics. Rows are contextual rather than matched ablations; full matched controls are in the supplement.

Benchmark	System / core model	Score
DiscoveryBench	Reflexion / GPT-4 family	24.5 HMS
DiscoveryBench	RG-HLI/ GPT-4o-mini	28.09 HMS
NewtonBench	Vanilla agent / GPT-5	75.9% SA
NewtonBench	Vanilla agent / o4-mini	47.8% SA
NewtonBench	RG-HLI/ GPT-4o-mini	49.7% SA
UltraHorizon	Best free-step LLM / reported best	14.33
UltraHorizon	RG-HLI strict agent-only / GPT-4o-mini	51.04

5.3 Full-run mechanism evidence

The headline table alone would under-specify the central novelty: executable probes could improve a fixed language without showing language growth. Table 6 therefore isolates complete-run mechanism controls that remove typed contracts, residual handling, or protocol support.

Table 6: Full-run mechanism controls under native metrics.

Condition	Result	Mechanism isolated
DB fixed-language-none	0.68 HMS	contracts, slots, residual renderers removed
DB full RG-HLI	28.09 HMS	full hypothesis-language state and renderer
NB chart-control	47.5% SA	residual-born chart recombination removed
NB promotion-gate-off	49.4% SA	held-out promotion gate disabled
NB audited RG-HLI	49.7% SA	full residual-gated symbolic induction
UH all-off strict	0.00	induction and protocol support removed
UH strict agent-only	51.04	frozen agent without terminal controller
UH full RG-HLI	75.36	terminal protocol plus induced evidence rendering

Table 6 clarifies both contribution and limitation. DiscoveryBench collapses without typed contracts and renderers. NewtonBench is less sensitive to one gate, suggesting that active probing and symbolic verification dominate. UltraHorizon separates reasoning from protocol completion: strict agent-only behavior is nontrivial, but the full score requires satisfying the public terminal contract. Backbone scaling should therefore freeze the induced language and vary only the model.

5.4 What the full runs show

The full runs support a mechanism-level rather than universal-dominance claim. Typed residuals help when failures can be localized into missing slots, coordinates, validators, or renderers. They are weaker when exploration or semantic rendering is the main bottleneck. Detailed residual taxonomies, protocol controls, and worked traces are given in the supplement.

6 Limitations

RG-HLI remains limited by operator coverage. If the missing object cannot be represented as a contract, typed hole, validator, renderer, or residual-induced operator, ranking and verification cannot recover it by themselves. DiscoveryBench still depends on semantic rendering and multi-file assembly; NewtonBench has abstentions from missing coordinate charts; and UltraHorizon grid tasks reveal exploration failures. The current mechanism is therefore disciplined adaptation, not a proof of unbounded self-improvement. Its energy weights are also fixed rather than learned; future work should calibrate evidence, coverage, validation loss, and complexity on held-out residual classes.

The practical implication is that inference-time improvement should not be measured only by longer traces or larger search trees. In scientific tasks, the useful object is often a typed intermediate representation: a role assignment, a coordinate transform, a validator, a renderer, or a residual class. RG-HLI makes these objects explicit and persistent while keeping the final benchmark answer separated from the internal artifact. This separation is what makes the method testable: ablations can remove contracts, residuals, gates, or protocol support without changing the benchmark evaluator.

This creates two reporting obligations. First, scaling experiments should vary the backbone while freezing the induced language. If GPT-4o or DeepSeek improves mainly

by filling the same typed holes more accurately, the contribution is architectural amplification. If the stronger model induces qualitatively new residual operators, the contribution becomes language growth. Both outcomes are scientifically meaningful, but they should be reported separately. This is why the supplement reserves a matched backbone table rather than mixing model scaling with Direct, ReAct, and CodeAct baselines. Second, a paper should report not only the final score, but the state that was frozen before evaluation, the residual classes observed after evaluation, the operators admitted before the run, and the operators rejected by the promotion gate. These records make it possible to distinguish genuine language growth from accidental benchmark repair. They also make negative results useful: a recurring residual that no current operator closes is a precise description of the next missing scientific capability. In this sense, RG-HLI turns benchmark evaluation into an audit of the hypothesis language itself.

A further limitation is evaluator heterogeneity. DiscoveryBench rewards semantic agreement with a target hypothesis, NewtonBench rewards symbolic equivalence, and UltraHorizon rewards valid long-horizon environment behavior. These metrics are intentionally different, so a single aggregate number would be misleading. The cross-benchmark claim is therefore not that one scalar score improves everywhere, but that the same hypothesis-state interface can expose different failure objects under different evaluators. This also limits direct comparisons to agent baselines whose budgets, tool interfaces, or stopping rules differ from ours. The cleanest comparison is the matched protocol in which the backbone, split, evaluator, and frozen language are all specified.

A final practical limitation is that residual logs are useful only when they are inspectable. We therefore treat row-level artifacts, prompt contracts, validation traces, and rejected operators as part of the experimental result, not as debugging byproducts. Without these records, a high score could be caused by a lucky renderer, a hidden protocol assumption, or an accidental interaction with a judge. With them, reviewers can trace which failures became state, which residual classes remained unresolved, and whether the same frozen language was used across benchmark rows. Future versions should report failed operator inductions with the same care as successful language additions.

7 Conclusion

RG-HLI externalizes small-model scientific reasoning as typed, executable, residual-guided hypothesis-language growth. Across three benchmarks, one frozen state and operator interface supports tabular discovery, symbolic law induction, and long-horizon rule discovery. The central claim is deliberately narrow: the system does not guarantee open-ended self-improvement, but it makes failures formal objects that can be tested, compressed, audited, and eventually replaced by better operators. This turns scientific-agent evaluation from a comparison of opaque traces into a comparison of induced languages: what evidence they admit, what failures they remember, what operators they add, and which artifacts they can validate under a frozen protocol.

References

- Agarwal, D.; Majumder, B. P.; Adamson, R.; Chakravorty, M.; Gavireddy, S. R.; Parashar, A.; Surana, H.; Dalvi Mishra, B.; McCallum, A.; Sabharwal, A.; and Clark, P. 2025. AutoDiscovery: Open-Ended Scientific Discovery via Bayesian Surprise. *arXiv preprint arXiv:2507.00310*.
- Chen, T.; Anumasa, S.; Lin, B.; Shah, V.; Goyal, A.; and Liu, D. 2025. Auto-Bench: An Automated Benchmark for Scientific Discovery in LLMs. *arXiv preprint arXiv:2502.15224*.
- Chen, W.; Ma, X.; Wang, X.; and Cohen, W. W. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*.
- Cranmer, M. 2023. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*.
- Ellis, K.; Wong, C.; Nye, M.; Sable-Meyer, M.; Cary, L.; Morales, L.; Hewitt, L.; Solar-Lezama, A.; and Tenenbaum, J. B. 2021. DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*.
- Gao, L.; Madaan, A.; Zhou, S.; Alon, U.; Liu, P.; Yang, Y.; Callan, J.; and Neubig, G. 2023. PAL: Program-Aided Language Models. In *International Conference on Machine Learning*.
- Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; Joshi, T. T.; Moazam, H.; Miller, H.; Zaharia, M.; and Potts, C. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714*.
- Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*.
- Luo, H.; Zhang, H.; Zhang, X.; Wang, H.; Qin, Z.; Lu, W.; Ma, G.; He, H.; Xie, Y.; Zhou, Q.; Hu, Z.; Mi, H.; Wang, Y.; Tan, N.; Chen, H.; Fung, Y. R.; Yuan, C.; and Shen, L. 2025. UltraHorizon: Benchmarking Agent Capabilities in Ultra Long-Horizon Scenarios. *arXiv preprint arXiv:2509.21766*.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*.
- Majumder, B. P.; Surana, H.; Agarwal, D.; Dalvi Mishra, B.; Meena, A.; Prakhar, A.; Vora, T.; Khot, T.; Sabharwal, A.; and Clark, P. 2024. DiscoveryBench: Towards Data-Driven Discovery with Large Language Models. *arXiv preprint arXiv:2407.01725*.
- Pearl, J. 2009. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition.
- Peters, J.; Bühlmann, P.; and Meinshausen, N. 2016. Causal Inference by Using Invariant Prediction: Identification and Confidence Intervals. *Journal of the Royal Statistical Society: Series B*, 78(5): 947–1012.
- Rainforth, T.; Foster, A.; Ivanova, D. R.; and Bickford Smith, F. 2023. Modern Bayesian Experimental Design. *arXiv preprint arXiv:2302.14545*.
- Romera-Paredes, B.; Barekatin, M.; Novikov, A.; Balog, M.; Kumar, M. P.; Dupont, E.; Ruiz, F. J. R.; Ellenberg, J. S.; Wang, P.; Fawzi, O.; Kohli, P.; and Fawzi, A. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature*, 625(7995): 468–475.
- Schick, T.; Dwivedi-Yu, J.; Dessi, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*.
- Udrescu, S.-M.; and Tegmark, M. 2020. AI Feynman: A Physics-Inspired Method for Symbolic Regression. *Science Advances*, 6(16).
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023a. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*.
- Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; and Ji, H. 2024. Executable Code Actions Elicit Better LLM Agents. In *International Conference on Machine Learning*.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023b. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations*.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.
- Yamada, Y.; Lange, R. T.; Lu, C.; Lu, C.; Hu, S.; Foerster, J.; Ha, D.; and Clune, J. 2026. Towards End-to-End Automation of AI Research. *Nature*.
- Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.
- Zelikman, E.; Wu, Y.; Mu, J.; and Goodman, N. D. 2022. STaR: Bootstrapping Reasoning With Reasoning. *arXiv preprint arXiv:2203.14465*.
- Zheng, T.; Tam, K. K.-W.; Nguyen, N. H.-N. K.; Xu, B.; Wang, Z.; Cheng, J.; Tsang, H. T.; Wang, W.; Bai, J.; Fang, T.; Song, Y.; Wong, G. Y.; and See, S. 2025. NewtonBench:

Benchmarking Generalizable Scientific Law Discovery in LLM Agents. *arXiv preprint arXiv:2510.07172*.

Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; and Wang, Y.-X. 2024a. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *International Conference on Machine Learning*.

Zhou, D.; Schärli, N.; Hou, L.; Wei, J.; Scales, N.; Wang, X.; Schuurmans, D.; Cui, C.; Bousquet, O.; Le, Q. V.; and Chi, E. H. 2023. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *International Conference on Learning Representations*.

Zhou, P.; Pujara, J.; Ren, X.; Chen, X.; Cheng, H.-T.; Le, Q. V.; Chi, E. H.; Zhou, D.; Mishra, S.; and Zheng, H. S. 2024b. Self-Discover: Large Language Models Self-Compose Reasoning Structures. *arXiv preprint arXiv:2402.03620*.